

SAÉ 2.01-2.02 : Rapport POO

Version 1

Yann Renard
Tom Cox
Alexandre Sorel

27 mai 2026

Table des matières

1 Version 1 : Un seul moyen de transport	3
1.1 Lancement et utilisation	3
1.2 Diagramme UML et mécanismes objets	3
1.3 Analyse technique des fonctionnalités	5
1.4 Analyse des tests	5

1 Version 1 : Un seul moyen de transport

1.1 Lancement et utilisation

L'application est développée avec Java 21 et Maven. Le code source est disponible sur le dépôt GitLab de l'équipe E5. Pour lancer l'application, il faut d'abord compiler le projet avec Maven, puis exécuter la classe principale.

Compilation et exécution depuis la racine du projet :

```
1 mvn compile
2 mvn exec:java
   -Dexec.mainClass="sae.transport.comparison.AppTerminal"
```

Les fichiers de données CSV doivent être placés dans `src/main/resources/`. La bibliothèque graphes (`sae_s2_2026.jar`, `jgrapht-core-1.5.1.jar`, `jheaps-0.14.jar`) est référencée dans le dossier `lib/` à la racine du projet et déclarée dans le `pom.xml`.

1.2 Diagramme UML et mécanismes objets

Tableau 1 : Classes de la Version 1

Classe/Enum	Package	Rôle
TypeCout	models	Enum représentant les 3 critères : TEMPS, PRIX, CO2
Cout	models	Stocke les 3 valeurs de coût dans une <code>Map<TypeCout, Double></code>
Ville	models	Implémente <code>Lieu</code> (lib IUT). Représente une ville du réseau
Trajet	models	Implémente <code>Connexion</code> (lib IUT). Connexion entre deux <code>Ville</code> avec un <code>Cout</code> et une <code>ModaliteTransport</code>
Voyageur	models	Représente l'utilisateur : nom et <code>TypeCout</code> (critère à optimiser)
Plateforme	models	Classe centrale : contient les listes de <code>Ville</code> et <code>Trajet</code> , expose les fonctionnalités de chargement, filtrage et tri

Table 1.1 : Classes de la Version 1

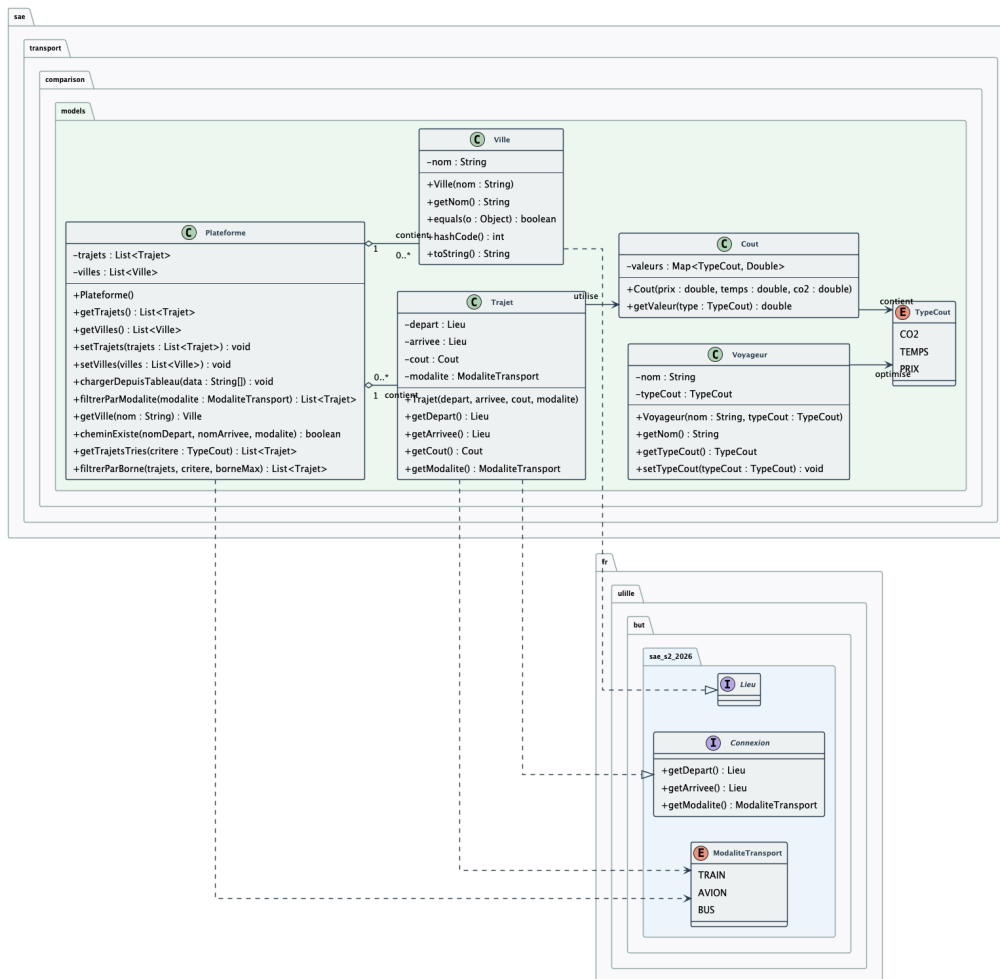


Figure 1.1 : Diagramme UML — Version 1

Mécanismes objets mis en œuvre

Interfaces et implémentation : Les classes `Ville` et `Trajet` implémentent les interfaces `Lieu` et `Connexion` fournies par la bibliothèque de l'IUT. Ce choix permet à notre code d'être directement compatible avec la bibliothèque de graphes, qui attend des objets `Lieu` et `Connexion`. C'est le principe de programmation par interface : on dépend d'une abstraction plutôt que d'une implémentation concrète.

Énumérations : `TypeCout` et `ModaliteTransport` (fournie par la lib) sont des enums. Ce mécanisme est approprié ici car les valeurs possibles sont fixes et connues à l'avance. Utiliser une enum plutôt qu'une `String` évite les erreurs de frappe et rend le code plus lisible.

Collections : La classe `Plateforme` utilise des `List<Ville>` et `List<Trajet>` pour stocker le réseau. La classe `Cout` utilise une `Map<TypeCout, Double>` pour associer chaque type de coût à sa valeur. Ce choix permet d'accéder à n'importe quel critère avec `getValeur(TypeCout)`.

Encapsulation : Tous les attributs sont privés. L'accès se fait uniquement via des getters, ce qui protège l'état interne des objets et permet de modifier l'implémentation sans impacter le code appelant.

1.3 Analyse technique des fonctionnalités

Tableau 2 : Correspondance fonctionnalités / méthodes

Méthode	Fonctionnalité couverte (sujet)
<code>chargerDepuisTableau(String[])</code>	Vérifier la validité des données et reconstituer le réseau
<code>filtrerParModalite(ModaliteTransport)</code>	Filtrer les données sur un moyen de transport spécifique
<code>getVille(String)</code>	Utilitaire interne pour retrouver une ville par son nom
<code>cheminExiste(...)</code>	Déterminer s'il existe un voyage possible entre deux villes
<code>getTrajetsTries(TypeCout)</code>	Afficher les meilleurs voyages ordonnés selon le critère
<code>filtrerParBorne(...)</code>	Exclure les alternatives qui excèdent les bornes définies

Table 1.2 : Correspondance fonctionnalités / méthodes

La méthode `chargerDepuisTableau` effectue la validation des données en vérifiant que chaque ligne contient exactement 6 colonnes et que tous les coûts sont positifs ou nuls. En cas d'erreur, une `IllegalArgumentException` est levée (remplacée par `DonneesInvalidesException` en Version 2).

La recherche des meilleurs itinéraires repose sur la bibliothèque de graphes : on construit un `MultiGrapheOrienteValue` dont les sommets sont les `Ville` et les arêtes les `Trajet` valuées selon le critère choisi, puis on appelle `AlgorithmeKPCC.kpcc` pour obtenir les k plus courts chemins.

1.4 Analyse des tests

Deux classes de test ont été développées avec JUnit 5 : `CoutTest` et `PlateformeTest`.

`CoutTest` couvre :

- La création d'un `Cout` avec des valeurs valides
- La récupération correcte de chaque valeur (PRIX, TEMPS, CO2)

- Le cas limite avec des valeurs à zéro

PlateformeTest couvre :

- Le chargement correct des données : vérification du nombre de villes (4) et de trajets (6)
- La détection des données invalides : ligne incomplète, coûts négatifs
- Le filtrage par modalité : vérification du nombre de trajets TRAIN et AVION
- La recherche de ville : ville existante et ville inexistante (**null**)
- La vérification d'existence de chemin : chemin existant, chemin inexistant, ville inconnue
- Le tri par critère : vérification que le premier élément est bien le moins coûteux
- Le filtrage par borne : vérification que tous les trajets retournés respectent la limite

Ces tests couvrent les cas simples, les cas limites et les cas d'erreur pour chaque fonctionnalité principale de cette version.